SSH Settings Lab (90 minutes)

Roles: On **your** VM, you are the **owner**: you change settings and watch the logs. On your **partner's** VM, you are the **tester**: you try to log in and report what happened. Every exercise, you do both at once.

Time	Part
0:00	Setup
0:15	Exercise 1: Logging
0:25	Exercise 2: Root
0:30	Exercise 3: Who is allowed in
0:45	Exercise 4: Keys
1:00	Exercise 5: The password trap
1:20	Wrap-up and cleanup

Setup (owner)

1. Make sure the SSH server is running. (The checking command you'll use later only works while it's running.)

   ```
   sudo systemctl start ssh
   ```

2. Check what's in the settings folder:

   ```
   ls /etc/ssh/sshd_config.d/
   ```

   If any `.conf` files are listed, turn each one off by renaming it. For example:

   ```
   sudo mv /etc/ssh/sshd_config.d/10-allowgroups.conf /etc/ssh/sshd_config.d/10-allowgroups.conf.off
   ```

   Only files ending in `.conf` are read.

3. Create the lab group and lab user. Give `labuser` a simple password and tell your partner what it is.

   ```
   sudo groupadd sshlab
   ```

   ```
   sudo adduser labuser
   ```

```
sudo usermod -aG sshlab labuser
```

You also need the SSH user you made for homework. If you don't have one, make it now with `sudo adduser`, but **don't** add it to `sshlab`.

4. Create the lab settings file. The name starts with `05` so SSH reads it first, which means its settings win.

```
sudoedit /etc/ssh/sshd_config.d/05-lab.conf
```

Type this in exactly, then save:

```
LogLevel QUIET
#AllowGroups sshlab
PasswordAuthentication yes
KbdInteractiveAuthentication yes
PermitRootLogin yes
```

This file is **insecure on purpose**. You'll fix it during the lab.

5. Apply it, then give your partner your IP address:

```
sudo sshd -t && sudo systemctl restart ssh
```

```
ip -br a
```

The change routine

You'll do this after every change:

1. Edit: `sudoedit /etc/ssh/sshd_config.d/05-lab.conf`
2. Check and apply: `sudo sshd -t && sudo systemctl restart ssh`
3. Confirm: `sudo sshd -T | grep -i <setting name>`

> **NOTE:** `sshd -T` reads the **file**, not the running server. If you skip step 2, `-T` shows your new value but SSH still behaves the old way.

Exercise 1: Logging (`LogLevel`)

1. **Owner:** open a second terminal and watch the SSH log:

```
sudo journalctl -u ssh -f
```

2. **Tester:** log in as `labuser` with the password, then type `exit`. Then try again and type the password **wrong** on purpose.

The first time you connect to a machine, SSH asks whether you trust it. Type `yes`.

3. **Owner:** what showed up in the log? Hint: lines that start with `pam_unix` come from the password-checking library, not from SSH itself. Did SSH log anything of its own?
4. **Owner:** change `LogLevel QUIET` to `LogLevel VERBOSE` and apply it. **Tester:** repeat step 2.
5. **Owner:** find the SSH lines for the successful login (`Accepted ...`) and the failed one (`Failed ...`). What do they tell you that the `pam_unix` lines didn't?

Think about it: an attacker who gets root might set `LogLevel QUIET`. How would you notice?

Exercise 2: Root (`PermitRootLogin`)

1. **Tester:** try `ssh root@<partner IP>` and guess some passwords. The setting says `yes`, so why can't you get in?
2. **Owner:** find root's failed attempts in the log. Then run this. What does the `L` mean?

```
sudo passwd -S root
```

3. **Owner:** set `PermitRootLogin no` anyway and apply it. Why bother, if root is already locked?

Exercise 3: Who is allowed in (`AllowGroups`)

1. **Tester:** log in as your partner's **homework user** with its password. It should work.
2. **Owner:** remove the `#` in front of `AllowGroups sshlab` and apply it.
3. **Tester:** try the homework user again, then try `labuser`.
4. **Owner:** find the log line that explains why the homework user was refused.
5. **Owner:** add the homework user to the group. **Tester:** try again. Did the owner need to restart SSH? Why or why not?

```
sudo usermod -aG sshlab <homework user>
```

6. **Owner:** take the user back out of the group:

```
sudo gpasswd -d <homework user> sshlab
```

Notice: the tester only sees "Permission denied." The owner's log says *why*. Attackers get less information than defenders do.

Exercise 4: Keys

1. **Tester:** make a key on your own VM. Give it a passphrase.

```
ssh-keygen -t ed25519
```

2. **Tester:** copy it to your partner's `labuser`. This asks for `labuser`'s password one last time.

```
ssh-copy-id labuser@<partner IP>
```

3. **Tester:** log in as `labuser`. You might be asked for your **key's passphrase**, but not `labuser`'s password. What's the difference?

   On Ubuntu Desktop, the passphrase question may pop up as a window instead of in the terminal, and it may offer to remember the passphrase for you.

4. **Owner:** in the log, find the `Accepted publickey` line and read out the `SHA256:...` value. **Tester:** compare it with your key's fingerprint:

```
ssh-keygen -lf ~/.ssh/id_ed25519.pub
```

Exercise 5: The password trap

Goal: stop **all** password logins, while keys keep working.

1. **Owner:** set `PasswordAuthentication no` and apply it.

2. **Tester:** try a password login. The `-o` option tells your SSH program not to use your key, so you're testing passwords only.

```
ssh -o PubkeyAuthentication=no labuser@<partner IP>
```

   Did you get in?

3. **Tester:** run it again with `-v` added, and press **Ctrl+C** at the password prompt. Scroll up and find the line `Authentications that can continue:`. What does it list?

4. **Owner:** figure out which setting is still letting passwords in, change it, and apply it. **Tester:** test again.

5. Together, try all four combinations and fill in this table. **Tester:** keep the `-o PubkeyAuthentication=no` option on every try, or your key from Exercise 4 will log you in and spoil the result.

`PasswordAuthentication`	`KbdInteractiveAuthentication`	**Password login works?**	**"can continue" list**
yes	yes		
no	yes		

`PasswordAuthentication`	`KbdInteractiveAuthentication`	**Password login works?**	**"can continue" list**
yes	no		
no	no		

Look closely at the password prompt each time. Is it always worded the same way? When does it change?

6. Finish with both set to `no`. **Tester:** confirm your key login still works.

Stretch (if you finish early)

- **`MaxAuthTries`:** temporarily set `PasswordAuthentication yes` and `MaxAuthTries 2`. **Tester:** type wrong passwords. What happens, and what does the log say? Set both back afterward.
- **`Banner`:** put a warning in `/etc/issue.net` with `sudoedit`, then add `Banner /etc/issue.net`. **Tester:** log in and read it.
- **`sshd -T -C`:** ask SSH which settings would apply to one specific user, without logging in:

  ```
  sudo sshd -T -C user=labuser,host=x,addr=10.0.0.5 | grep -i passwordauth
  ```

  Here the answer is the same as plain `sshd -T`. It only differs when the settings include a `Match` block, which is how one server gives different rules to different users or networks.

Wrap-up questions

1. Which setting surprised you most?
2. You set `PasswordAuthentication no` and it didn't fully work. What habit catches mistakes like that?
3. Exercise 3 gave the tester much less information than the owner. Why is that good for defenders?

Cleanup (or keep it for next week)

1. Delete the lab settings file:

   ```
   sudo rm /etc/ssh/sshd_config.d/05-lab.conf
   ```

2. Rename any `.off` files back to `.conf`.
3. Remove the lab user and group:

   ```
   sudo deluser --remove-home labuser
   ```

```
sudo delgroup sshlab
```

4. Apply the change:

```
sudo systemctl restart ssh
```